

Clase 9

HEFESTO · Paso 4

Integración de Datos — y el cierre del ciclo

Las tablas dejan de estar vacías: del archivo a la base, limpio y trazable. Y de la base al dashboard que responde las preguntas del Paso 1.

TP #2 · Data Warehouse con metodología HEFESTO · Clase 4 de 4 · Caso guía: Ecobici CABA

Martes 29/09/2026 · 2 horas · Yago Gándara (JTP)

Checkpoint del Paso 3 (5')

Hoy se carga. Si alguna de estas tres cosas falta, la carga no arranca: resuélvanlo en los primeros minutos.

1

El modelo existe en su MySQL

SHOW TABLES muestra sus dim_ y su fact_, con las FK. Es el entrega_modelo_logico.sql corrido, sin errores.

2

El archivo en la base: hoy

Todavía no lo tienen y está bien: no era consigna. Es lo primero que hacemos hoy: una tabla stg_ con todas las columnas como VARCHAR y LOAD DATA. Si en la Clase 7 usaron pandas, también sirve (to_sql).

3

Las reglas del Paso 2 a mano

La sección de consistencia de entrega_fuente_datos.md: qué se normaliza, qué se descarta, qué se completa con “Sin dato”. Hoy eso se convierte en código.

Hoja de ruta de hoy



Lo que sale de hoy

- Los scripts de carga, numerados en el orden en que corren (01 → 06).
- entrega_informe.md: calidad de datos con números, verificación de la carga, política de actualización.
- El plan para el 06/10: dashboard.md y la presentación.

Cómo lo hacemos

- Paso a paso sobre Ecobici, con SQL puro en MySQL y los números reales de cada corrida. Lo mismo se puede hacer con pandas: cambia la herramienta, no las etapas.
- Al final vemos una implementación completa (de otro profe de la cátedra) para que sepan a qué apuntar.

Paso 4 en HEFESTO: Integración de Datos

“Una vez construido el modelo lógico, se deberá proceder a poblarlo con datos, utilizando técnicas de limpieza y calidad de datos, procesos ETL, etc.; luego se definirán las reglas y políticas para su respectiva actualización.”

a) Carga inicial

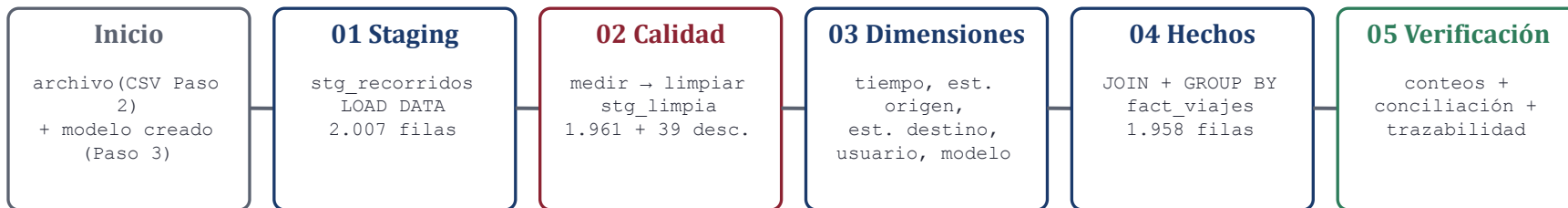
- “Se debe evitar que el DW sea cargado con valores faltantes o anómalos”: condiciones y restricciones para usar solo los datos de interés.
- “Primero se cargarán los datos de las dimensiones y luego los de las tablas de hechos”, respetando la correspondencia entre cada elemento.
- “Registrar en detalle las acciones llevadas a cabo”: qué hace el proceso en general y después cada paso, de lo general a lo específico.

b) Actualización

- “Cuando se haya cargado en su totalidad el DW, se deben establecer sus políticas y estrategias de actualización o refresco de datos.”
- Especificar las tareas de limpieza, calidad y ETL que se repiten en cada actualización.
- Especificar, general y detalladamente, qué hace cada software.
- Caso práctico del PDF: refresco diario; PRODUCTO y CLIENTE se recargan completas; FECHA incremental; los hechos de los últimos 30 días se reemplazan.

El proceso ETL del caso guía, paso a paso

Seis scripts SQL, numerados en el orden en que corren. Cada uno es un “paso” del proceso; la flecha es la “relación”. Esto mismo, con sus nombres, es la primera sección de su entrega_informe.md.



06 Actualización: el mismo camino 01 → 05, con tres diferencias: se vacía staging, las dimensiones se actualizan (total o incremental según la tabla) y los hechos del mes se borran y se recargan.

Por qué scripts numerados

El PDF pide explicar “qué hace el proceso en general y luego qué hace cada paso”. Un archivo por paso, con un comentario arriba que dice qué hace y qué regla del Paso 2 aplica, ES esa documentación. Y se puede volver a correr desde cero.

Dónde queda cada cosa

Todo dentro de la misma base (ecobici_dw): las tablas stg_ son el área de trabajo; las dim_ y fact_ son el modelo. La guía pide exactamente eso: un punto intermedio donde revisar y limpiar antes de tocar las tablas finales.

01 · Staging: el archivo entra tal cual

```
SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ecobici_dw;
CREATE TABLE stg_recorridos (
  id_recorrido          VARCHAR(20),
  duracion_recorrido    VARCHAR(20),
  fecha_origen_recorrido VARCHAR(30),
  ...                   -- las 18 columnas, TODAS VARCHAR
  edad_usuario          VARCHAR(10)
) ENGINE=InnoDB;
LOAD DATA LOCAL INFILE 'recorridos_ecobici_2026-08_muestra.csv'
INTO TABLE stg_recorridos
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'      -- trampa 1: el archivo es CRLF
IGNORE 1 LINES;
SELECT COUNT(*) FROM stg_recorridos;  --> 2007
```

Todo VARCHAR, a propósito

En staging no se interpreta nada: si una edad viene vacía o una duración negativa, entra igual. Recién en 02 se decide qué hacer con eso. Si el LOAD DATA falla por tipos, la culpa es del CSV y todavía no sabemos dónde.

Las tres trampas

- CRLF: sin LINES TERMINATED BY '\r\n', la última columna arrastra un \r.
- Collation: para ver variantes de texto, comparar con BINARY.
- Charset del cliente: SET NAMES utf8mb4 o "Pacífico" se guarda como "Pacífico".

Si usan pandas: el equivalente es `pd.read_csv(..., dtype=str, keep_default_na=False)` y `df.to_sql('stg_recorridos', ...)`. Misma idea: primero adentro, después limpio. Con MySQL 8 hay que habilitar `local_infile=1` en el servidor y en el cliente (`--local-infile=1`).

02 · Calidad de datos (1/2): medir antes de tocar

Cada regla sale de la sección “consistencia” del Paso 2. Primero se cuenta cuánto hay de cada problema — con SQL sobre staging — y ese número va al informe.

Chequeo (SQL sobre stg_recorridos)	Encontrado	Regla del Paso 2	Acción en 02
id_recorrido duplicados	7 filas de más	filas idénticas → una sola	GROUP BY en la subconsulta
duracion_recorrido ≤ 0	23	imposible → descartar e informar	motivo_descarte = 'duracion <= 0'
duracion_recorrido > 24 h	16 (máx. 225.505 s)	imposible → descartar e informar	motivo_descarte = 'duracion > 24 h'
edad_usuario vacía	55	→ rango 'Sin dato'	CASE ... ELSE 'Sin dato'
genero_usuario: variantes / vacíos	7 escrituras / 40 vacíos	mapeo manual a 4 valores	CASE UPPER(TRIM(...))
nombre de estación: escrituras	54 para 18 códigos	nombre por código, el más frecuente	se resuelve en 03 (dimensiones)
fecha_destino_recorrido vacía	44	no se usa (Paso 2)	ninguna

Regla de oro: la decisión ya está tomada (Paso 2); hoy solo se ejecuta y se cuenta. Si aparece un problema nuevo que el Paso 2 no vio, se agrega a la tabla del informe con su regla — no se arregla en silencio.

02 · Calidad de datos (2/2): stg_limpia, nada se borra

```
CREATE TABLE stg_limpia AS
SELECT CAST(id_recorrido AS UNSIGNED) AS id_recorrido,
       CAST(duracion_recorrido AS SIGNED) AS duracion_seg,
       DATE(STR_TO_DATE(fecha_origen_recorrido, '%Y-%m-%d %H:%i:%s')) AS fecha,
       CASE WHEN HOUR(...) < 6 THEN 'madrugada' WHEN HOUR(...) < 12 THEN 'mañana'
            WHEN HOUR(...) < 18 THEN 'tarde' ELSE 'noche' END AS franja_horaria,
       CAST(id_estacion_origen AS UNSIGNED) AS codigo_estacion_origen,
       CAST(id_estacion_destino AS UNSIGNED) AS codigo_estacion_destino,
       CASE UPPER(TRIM(genero_usuario))
            WHEN 'MALE' THEN 'Masculino' WHEN 'M' THEN 'Masculino'
            WHEN 'FEMALE' THEN 'Femenino' WHEN 'F' THEN 'Femenino'
            WHEN 'OTHER' THEN 'Otro' ELSE 'Sin dato' END AS genero,
       CASE WHEN edad_usuario = '' THEN 'Sin dato' -- regla P2: edad
            WHEN CAST(edad_usuario AS UNSIGNED) <= 25 THEN '18-25'
            ... ELSE '60+' END AS rango_etario,
       UPPER(TRIM(modelo_bicicleta)) AS modelo,
       CASE WHEN CAST(duracion_recorrido AS SIGNED) <= 0 THEN 'duracion <= 0'
            WHEN CAST(duracion_recorrido AS SIGNED) > 86400 THEN 'duracion > 24 h'
            ELSE NULL END AS motivo_descarte
FROM (SELECT * FROM stg_recorridos GROUP BY id_recorrido, ...) s; -- dedup
ALTER TABLE stg_limpia ADD PRIMARY KEY (id_recorrido);
```

Resultado (real)

- 2.007 filas → 2.000 únicas.
- 1.961 se cargan.
- 23 con motivo 'duracion <= 0'.
- 16 con motivo 'duracion > 24 h'.

motivo_descarte = trazabilidad

Las filas descartadas siguen en stg_limpia con su motivo. Nunca se hace DELETE del archivo: cualquier número del DW se puede bajar hasta el id_recorrido, y cualquier fila excluida se puede explicar.

03 · dim_tiempo: se genera, no se extrae

El PDF carga FECHA con un procedimiento que recorre las fechas entre Fecha_Desde y Fecha_Hasta y calcula cada campo (fig. 5.32). Acá es lo mismo en una consulta: todos los días del rango × las 4 franjas.

```
INSERT INTO dim_tiempo (fecha, franja_horaria, dia, dia_semana, nro_dia_semana,
                        mes, nombre_mes, trimestre, anio)
SELECT d.fecha, f.franja, DAY(d.fecha),
       ELT(WEEKDAY(d.fecha)+1,'lunes',...,'domingo'), WEEKDAY(d.fecha)+1,
       MONTH(d.fecha), ELT(MONTH(d.fecha),'enero',...,'diciembre'),
       QUARTER(d.fecha), YEAR(d.fecha)
FROM (SELECT DATE_ADD((SELECT MIN(fecha) FROM stg_limpia), INTERVAL seq DAY)
      AS fecha FROM seq_0_to_366) d    -- MariaDB; en MySQL 8: WITH RECURSIVE
JOIN (SELECT 'madrugada' AS franja UNION SELECT 'mañana'
      UNION SELECT 'tarde' UNION SELECT 'noche') f
WHERE d.fecha <= (SELECT MAX(fecha) FROM stg_limpia);
--> 31 días x 4 franjas = 124 filas
```

Por qué generarla

- Un día sin viajes también existe: si la dimensión sale del archivo, ese día desaparece del calendario y el gráfico mensual miente.
- Los nombres (lunes, agosto) se calculan una vez acá, no en cada consulta del dashboard.
- Fecha_Desde / Fecha_Hasta: las mismas variables del proceso del PDF.

Con pandas: `pd.date_range(min, max) × 4 franjas → DataFrame → to_sql('dim_tiempo', if_exists='append')`.

03 · Dimensiones (2/2): valores únicos + clave nueva

```
-- dim_estacion_origen: nombre por código, escritura MÁS FRECUENTE (regla P2)
INSERT INTO dim_estacion_origen (codigo_estacion, nombre_estacion)
SELECT codigo, nombre FROM (
    SELECT CAST(id_estacion_origen AS UNSIGNED) AS codigo,
           TRIM(nombre_estacion_origen) AS nombre,
           ROW_NUMBER() OVER (PARTITION BY CAST(id_estacion_origen AS UNSIGNED)
                              ORDER BY COUNT(*) DESC) AS rn
    FROM stg_recorridos GROUP BY codigo, nombre
) x WHERE rn = 1; --> 18 filas
-- dim_estacion_destino: misma consulta sobre las columnas _destino --> 18
-- dim_usuario: combinaciones que existen en los datos limpios
INSERT INTO dim_usuario (rango_etario, genero)
SELECT DISTINCT rango_etario, genero FROM stg_limpia; --> 23 (de 24 posibles)
INSERT INTO dim_modelo (nombre_modelo)
SELECT DISTINCT modelo FROM stg_limpia; --> 2
```

La clave nueva la pone MySQL

AUTO_INCREMENT genera id_estacion_origen 1..18 en el orden del INSERT. La clave del archivo (codigo_estacion) queda guardada al lado, con UNIQUE: es el puente para el JOIN del paso 04 y para volver al archivo.

Dos perspectivas, dos cargas

dim_estacion_origen se llena desde las columnas *_origen y dim_estacion_destino desde las *_destino. Si una estación solo aparece como destino en el mes, existe en una tabla y no en la otra: correcto, cada dimensión describe su perspectiva.

Orden dentro de 03

Da igual entre dimensiones (ninguna depende de otra en estrella). En copo de nieve el PDF pide “del nivel más general al más detallado”: primero PAISES, después PROVINCIAS... porque la FK apunta hacia arriba.

04 · Hechos: resolver claves y preagregar

“Se decidió aplicar una preagregación a los hechos [...] se utilizará la cláusula GROUP BY para agrupar todos los registros a través de las claves primarias de esta tabla.” (caso práctico del PDF, VENTAS)

```
INSERT INTO fact_viajes (id_tiempo, id_estacion_origen, id_estacion_destino,
                        id_usuario, id_modelo,
                        cantidad_viajes, duracion_total_min, duracion_promedio_min)
SELECT t.id_tiempo, eo.id_estacion_origen, ed.id_estacion_destino,
       u.id_usuario, m.id_modelo,
       COUNT(s.id_recorrido)           AS cantidad_viajes,      -- Cantidad de
viajes
       ROUND(SUM(s.duracion_seg)/60, 2) AS duracion_total_min,  -- Duración total
       ROUND(AVG(s.duracion_seg)/60, 2) AS duracion_promedio_min -- Duración
promedio
FROM stg_limpia s
JOIN dim_tiempo t   ON t.fecha = s.fecha AND t.franja_horaria = s.franja_horaria
JOIN dim_estacion_origen eo ON eo.codigo_estacion = s.codigo_estacion_origen
JOIN dim_estacion_destino ed ON ed.codigo_estacion = s.codigo_estacion_destino
JOIN dim_usuario u   ON u.rango_etario = s.rango_etario AND u.genero = s.genero
JOIN dim_modelo m    ON m.nombre_modelo = s.modelo
WHERE s.motivo_descarte IS NULL      -- solo los datos de interés
GROUP BY t.id_tiempo, eo.id_estacion_origen, ed.id_estacion_destino,
         u.id_usuario, m.id_modelo;
--> 1.958 filas de hechos · 1.961 viajes · 42.234,25 minutos
```

JOIN = correspondencia

Cada recorrido se “resuelve” contra cada dimensión por su clave natural (código, fecha+franja, rango+género, modelo) y sale con la clave subrogada. Si un JOIN pierde filas, falta un valor en la dimensión.

El GROUP BY es el grano

Las 5 claves del GROUP BY son la PK de fact_viajes = la granularidad del Paso 2. 1.961 viajes → 1.958 filas: grano fino, casi sin agregación. Con grano mensual serían decenas.

Verificar el JOIN antes de insertar: SELECT COUNT(*) del SELECT sin GROUP BY tiene que dar 1.961 (los limpios). Si da menos, un INNER JOIN está filtrando; si da más, una dimensión tiene duplicados.

05 · Verificación: el mismo número por dos caminos

Tabla	Filas
stg_recorridos (archivo)	2.007
stg_limpia (únicas)	2.000
stg_limpia descartadas	39
dim_tiempo	124
dim_estacion_origen	18
dim_estacion_destino	18
dim_usuario	23
dim_modelo	2
fact_viajes	1.958

```
SELECT 'archivo (limpio)' AS origen, COUNT(*) AS viajes,
       ROUND(SUM(duracion_seg)/60, 2) AS minutos
FROM stg_limpia WHERE motivo_descarte IS NULL
UNION ALL
SELECT 'DW (fact_viajes)', SUM(cantidad_viajes), SUM(duracion_total_min)
FROM fact_viajes;
```

```
+-----+-----+-----+
| archivo (limpio) | 1961 | 42234.43 |
| DW (fact_viajes) | 1961 | 42234.25 |
+-----+-----+-----+
```

Leer la conciliación

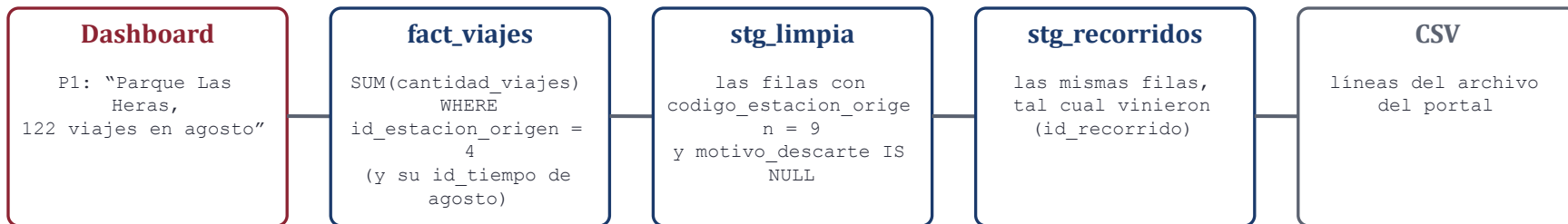
Viajes: $1.961 = 1.961$, ni perdidos ni duplicados. Minutos: 0,18 de diferencia sobre 42.234 (0,0004 %) por redondear a 2 decimales fila por fila. Se documenta y se acepta; si fueran 180, hay un JOIN roto.

Los tres chequeos mínimos de la guía

1) Conteo de filas por tabla. 2) Un total agregado en el DW = el mismo total calculado sobre el archivo (viajes, y un monto o duración). 3) Ninguna fila de hechos sin dimensión (LEFT JOIN ... IS NULL → 0; las FK ya lo garantizan, pero se muestra).

Trazabilidad: de un número del dashboard al archivo

Elegimos un número cualquiera y lo bajamos capa por capa. Si cada salto se puede mostrar con una consulta, el Data Mart está bien construido.



```

-- la fila de hechos con más viajes, y los recorridos del archivo que la componen (JOIN por claves naturales)
| id_tiempo | id_est_origen | id_est_destino | id_usuario | id_modelo | cantidad_viajes | duracion_total_min | |
| 67 | 17 | 4 | 14 | 2 | 2 | 85.62 |
| id_recorrido | fecha_hora_origen | duracion_seg | cod_origen | cod_destino | rango_etario | genero | modelo |
| 30000005 | 2026-08-17 12:36:14 | 4355 | 70 | 9 | 46-60 | Masculino | ICONIC |
| 30001654 | 2026-08-17 14:00:37 | 782 | 70 | 9 | 46-60 | Masculino | ICONIC |
  
```

$(4355 + 782) / 60 = 85,62$. Dos recorridos, un lunes 17/08 a la tarde, de Barrancas de Belgrano a Parque Las Heras. Cada clave subrogada se abre en su clave natural, y la clave natural está en el archivo. Esa cadena es lo que se evalúa.

b) Actualización: políticas y el proceso que las ejecuta

Aunque la carga del TP sea única, la guía pide definir cómo se actualizaría. Para Ecobici el portal publica un archivo por mes, así que la política se escribe como la del caso práctico del PDF, tabla por tabla:

Tabla	Estrategia	Por qué	En el PDF (caso práctico)
Frecuencia	mensual, al publicarse el archivo	es la cadencia de la fuente	“todos los días a las doce de la noche”
dim_estacion_origen / _destino	recarga total lógica: códigos nuevos se agregan, nombres se actualizan	las estaciones cambian de nombre y aparecen nuevas	PRODUCTO y CLIENTE “cargados totalmente cada vez”
dim_tiempo	incremental desde la última fecha	el calendario solo crece	FECHA “de manera incremental”
dim_usuario / dim_modelo	incremental (solo valores nuevos)	24 combinaciones posibles; un modelo nuevo se suma	—
fact_viajes	borrar y recargar el mes recibido	si el portal re-publica un mes corregido, se reemplaza entero	“el último mes será reemplazado cada vez”

```
SET @desde := '2026-08-01'; SET @hasta := '2026-08-31'; -- Fecha_Desde/Hasta
TRUNCATE TABLE stg_recorridos; LOAD DATA ... (archivo del mes)
INSERT INTO dim_estacion_origen (...) SELECT ...
ON DUPLICATE KEY UPDATE nombre_estacion = VALUES(nombre_estacion);
INSERT INTO dim_tiempo ... WHERE d.fecha > (SELECT MAX(fecha) FROM dim_tiempo);
DELETE f FROM fact_viajes f JOIN dim_tiempo t USING (id_tiempo)
WHERE t.fecha BETWEEN @desde AND @hasta;
INSERT INTO fact_viajes ... WHERE s.fecha BETWEEN @desde AND @hasta GROUP BY ...;
```

Corrido (real)

06_actualizacion.sql vuelto a correr con el mismo archivo: archivo 1.961 = DW 1.961, y las dimensiones no se duplican (UNIQUE + INSERT IGNORE / ON DUPLICATE KEY). Un proceso de actualización que se puede correr dos veces sin romper nada es un proceso bien hecho.

Consigna 4: scripts + entrega_informe.md

```
paso-4/
  01_staging.sql      02_calidad.sql      03_dimensiones.sql
  04_hechos.sql       05_verificacion.sql  06_actualizacion.sql
  (o un etl.py con funciones en ese orden)
# Paso 4 – Integración de datos
## Proceso general
_____ (qué hace cada paso, en orden; la figura de la slide 5)
## a) Carga inicial
### Calidad de datos
| Chequeo | Encontrado | Regla (Paso 2) | Acción | Filas |
| _____ | _____ | _____ | _____ | _____ |
### Carga de dimensiones (de dónde sale cada una; cuántas filas)
### Carga de hechos (JOIN por clave natural + GROUP BY al grano)
### Verificación
conteos por tabla · total archivo = total DW · trazabilidad de una fila
## b) Actualización
| Tabla | Estrategia (total/incremental/reemplazo) | Frecuencia | Por qué |
```

Se corrige con esto

- Los scripts corren de punta a punta, en orden, sobre una base vacía con el modelo del Paso 3.
- Hay un punto intermedio (staging) donde el archivo se puede revisar antes de tocar dim_/fact_.
- Cada problema de calidad tiene número, regla del Paso 2 y acción. Nada se arregla en silencio.
- Dimensiones con claves nuevas; hechos con JOIN por clave natural y GROUP BY al grano del Paso 2.
- Conteos + un total conciliado archivo vs DW + una fila trazada hasta el archivo.
- Política de actualización por tabla, con frecuencia y justificación.

Entrega final: dashboard.md, el ciclo se cierra

Por cada pregunta del Paso 1: la consulta SQL que la responde contra el modelo estrella (nunca contra staging ni el CSV), el resultado en tabla, un gráfico, y una frase que diga qué se ve.

```
-- P2. ¿Duración promedio por día de la semana y franja horaria?  
SELECT t.dia_semana, t.franja_horaria,  
       SUM(f.cantidad_viajes) AS viajes,  
       ROUND(SUM(f.duracion_total_min) / SUM(f.cantidad_viajes), 1)  
         AS duracion_promedio_min  
FROM fact_viajes f  
JOIN dim_tiempo t ON t.id_tiempo = f.id_tiempo  
GROUP BY t.nro_dia_semana, t.dia_semana, t.franja_horaria  
ORDER BY t.nro_dia_semana,  
         FIELD(t.franja_horaria, 'madrugada', 'mañana', 'tarde', 'noche');
```

El promedio, otra vez

duracion_promedio_min está en fact_viajes al grano día+franja+estaciones+perfil+modelo. Al agrupar por día de semana se RECALCULA total / cantidad: promediar la columna daría un promedio de promedios, y el número sería incorrecto.

1 · Pregunta

Textual, la del Paso 1.

2 · SQL

Sobre dim_ y fact_. Un JOIN por dimensión.

3 · Resultado

Tabla (las primeras filas si son muchas).

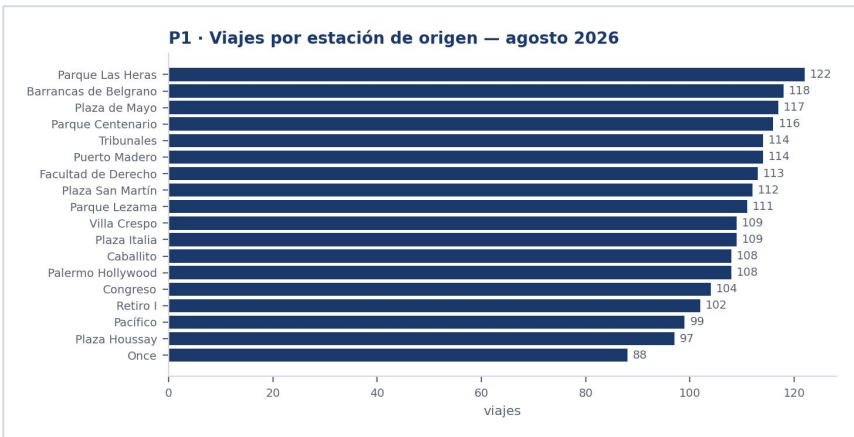
4 · Gráfico

El tipo que corresponda: barras, líneas, mapa de calor.

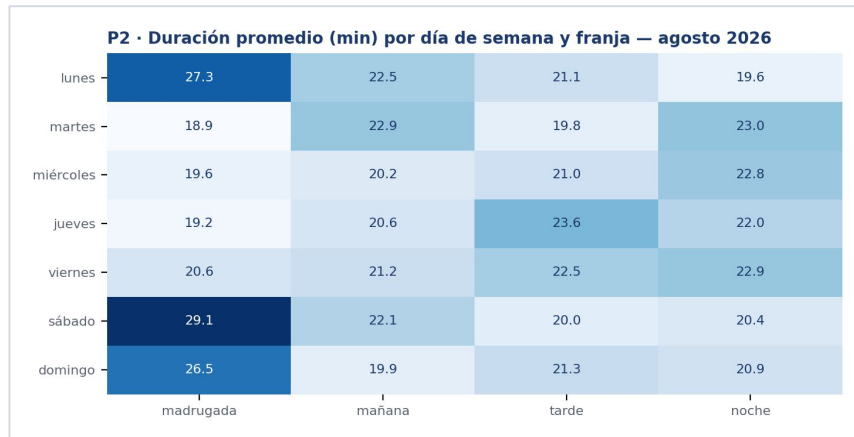
5 · Lectura

Una frase: qué responde, qué llama la atención.

Caso guía · así se responde una pregunta del Paso 1



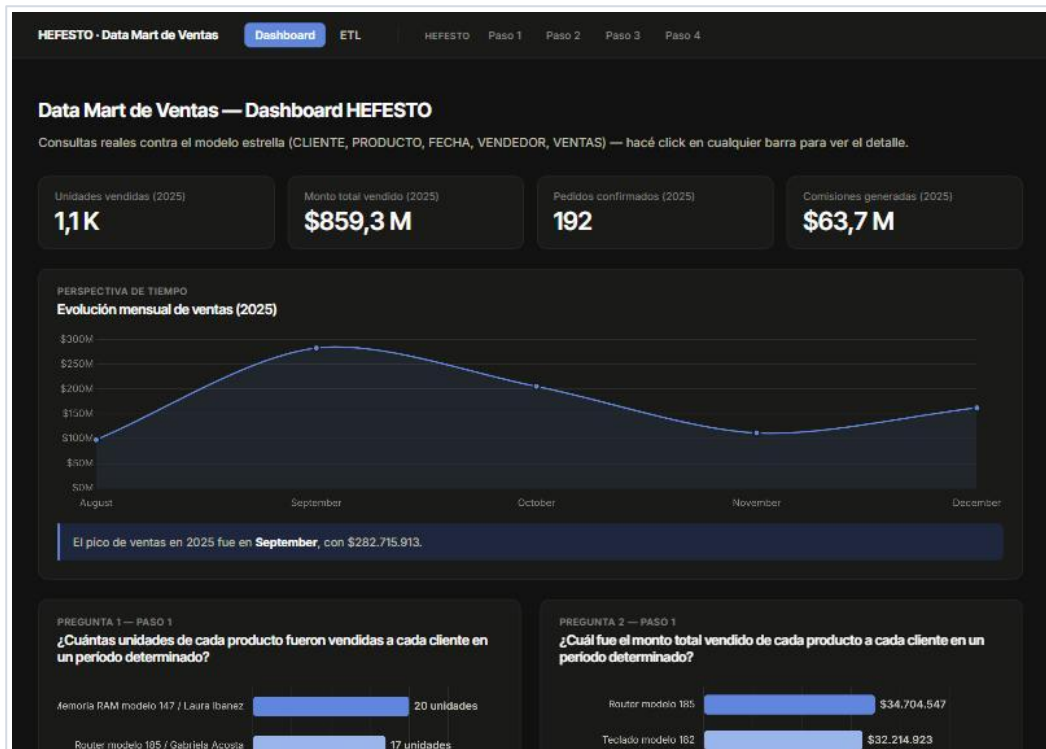
P1 · Lectura: el uso está muy repartido: la estación más usada (Parque Las Heras, 122) apenas supera en un 40 % a la menos usada (Once, 88). No hay una estación dominante en agosto.



P2 · Lectura: los viajes de madrugada del fin de semana son los más largos (29 min el sábado, 27 el lunes, 26 el domingo); en horario laboral el promedio se estabiliza en 20–23 min.

Se evalúa que el gráfico salga de la consulta sobre el modelo estrella y que la lectura sea honesta con los datos (si el resultado es “no hay diferencia”, se dice). Los dos gráficos de arriba son matplotlib sobre el resultado del SELECT.

Así se ve la meta: un TP2 completo



Lo recorreremos en vivo ahora. Si les interesa cómo está implementado, el repo se puede compartir.

Qué tiene (y ustedes también)

- Una tarjeta por pregunta del Paso 1: consulta real al modelo estrella, tabla, gráfico y una frase de lectura.
- Pestaña ETL: conteo de filas por tabla, conciliación archivo vs DW e historial de corridas. Es el Paso 4 en vivo.
- KPIs arriba y evolución mensual: la perspectiva Tiempo siempre visible.

Qué es opcional

Que sea una web hosteada (Render + base en Clever Cloud, ambos gratis), los cruces entre preguntas, el “click en la barra para ver el detalle”. Para la guía alcanza con [dashboard.md](#); esto muestra hasta dónde se puede llevar con las mismas 5 piezas.

La presentación del 06/10: qué vamos a mirar

1 El recorrido

Un minuto por paso: tema y preguntas (P1) → archivo y granularidad (P2) → modelo estrella (P3) → cómo se cargó (P4). Sin leer los .md: contarlos.

2 El dashboard

Cada pregunta del Paso 1 con su respuesta y su gráfico, en vivo o en dashboard.md. Una frase de lectura por pregunta.

3 La trazabilidad

Les vamos a pedir un número del dashboard y que lo bajen: consulta → fact_ → staging → archivo. Tengan las consultas a mano.

4 Las decisiones

Por qué ese esquema, por qué ese grano, qué se descartó y cuánto. Lo que quedó escrito en “Cambios respecto del Paso N” vale, no resta.

Tiempo por persona y orden: se confirman esta semana por el canal de la materia. Lo que no cambia: se entrega paso-4/ + dashboard.md antes de la clase, y se presenta con el Data Mart cargado en su MySQL (o hosteado, si eligieron ese camino).

Errores típicos del Paso 4



Limpiar el CSV a mano en Excel

No hay staging ni registro: nadie sabe qué se cambió. La limpieza es código que se puede volver a correr.



Cargar hechos antes que dimensiones

Las FK fallan, o peor: se cargan con claves inventadas. El orden del PDF es dimensiones → hechos.



JOIN que pierde filas sin que nadie lo note

Un INNER JOIN filtra silenciosamente. Contar antes y después; conciliar totales.



Gráficos hechos sobre el CSV

El dashboard debe salir del modelo estrella; si no, ¿para qué se construyó?



DELETE de las filas malas

Se pierde la trazabilidad. Se marcan con motivo y se excluyen con WHERE al cargar hechos.



Dimensión sacada del archivo con duplicados

18 estaciones con 54 nombres → 54 filas. Valores únicos por clave natural, con UNIQUE.



Promediar el promedio en el dashboard

AVG(duracion_promedio_min) por mes da un número incorrecto. SUM(total)/SUM(cantidad).



Informe sin números

“Se limpiaron los datos” no es un informe. 23 filas descartadas por duración ≤ 0 sí.

Ahora en clase (15') y hasta el 06/10

15'

Hoy, acá

- Escribir 02_calidad: la tabla “chequeo → encontrado → regla → acción” con SUS números, sobre su staging.
- Checkpoint conmigo: me muestran esa tabla. Si los números están, el resto es mecánico.

en casa

Esta semana

- 03 → 04 → 05 sobre su modelo. Correr todo desde cero al menos una vez.
- entrega_informe.md con los números reales y la política de actualización.
- dashboard.md: una pregunta por día y llegan.

06/10

Presentación

- Carpeta paso-4/ + dashboard.md entregados antes de la clase.
- Data Mart cargado y consultas a mano para la trazabilidad.

Si el JOIN pierde filas o el total no concilia

No lo tapen: al informe, con el número y la hipótesis. Una diferencia documentada vale más que un “da justo” sin mostrar cómo.

Con esto se cierra HEFESTO

Cuatro pasos, cuatro clases, un archivo que eligieron ustedes y que ahora responde preguntas.

Paso 1

Preguntas, indicadores, perspectivas, modelo conceptual.

Paso 2

El archivo real: columnas, indicadores conformados, granularidad, problemas detectados.

Paso 3

El modelo estrella: una dimensión por perspectiva, un hecho por indicador, DDL corrido.

Paso 4

Staging → calidad → dimensiones → hechos → verificación; política de actualización.

Dashboard

Cada pregunta del Paso 1 respondida con datos del modelo estrella, con gráfico y lectura.

Martes 06/10: presentación de los proyectos. Nos vemos con el Data Mart cargado.